TP 03 · Planificador de Rutas con Estrategia Greedy

Modalidad	TPO obligatorio
Trabajo	Grupos de hasta 4 integrantes
Lenguaje	Python 3.10
Robot	Unitree Go2 o G1
Duración de taller	2 horas
Preparación previa	Git + configuración del entorno (Sección 0)

0. Preparación del entorno de trabajo

Antes de comenzar con la implementación, cada grupo deberá descargar y verificar el entorno. Esta preparación se realiza una sola vez y se recomienda completarla antes del taller, para dedicar las 2 horas al desarrollo, las pruebas y el análisis del algoritmo.

Requisitos previos: Windows, Git instalado y Python 3.10 disponible desde la terminal.

1. Clonar el repositorio del laboratorio de UADE:

```
git clone https://github.com/tsamaan/UadeRobotLab.git
```

2. Ingresar a la carpeta del simulador:

```
cd UadeRobotLab\04Simuladores\UnitreeMujocoOficial
```

3. Ejecutar la instalación inicial. El script crea el entorno virtual, instala dependencias y prepara los SDK de Unitree:

```
setup_windows.bat
```

4. Verificar la configuración:

```
.\run_g1_sim.ps1 -SetupOnly
```

5. Iniciar el simulador G1:

```
.\run_g1_sim.ps1
```

También puede iniciarse con doble clic sobre abrir_g1_sim.bat.

6. Con el simulador abierto, ejecutar en otra terminal la prueba del laboratorio:

```
.\.venv\Scripts\python.exe .\examples\ejemplo_g1_simple.py
```

Si el ejemplo se ejecuta correctamente y el robot responde en MuJoCo, el entorno está listo para comenzar el TPO. El planificador greedy deberá probarse primero en simulación; cualquier demostración con robot físico será supervisada por el docente.

Repositorios de referencia: UADE Robot Lab: https://github.com/tsamaan/UadeRobotLab.git | Unitree Robotics: https://github.com/unitreerobotics

1. Objetivo del trabajo

Implementar en Python un planificador de movimientos para el robot Unitree utilizando una estrategia greedy. El objetivo es observar cómo una decisión localmente conveniente puede producir una buena ruta, una ruta no óptima o incluso un bloqueo aunque el mapa posea solución.

2. Consigna

1. Leer el mapa desde un archivo JSON provisto por la cátedra.
2. Obtener la grilla, la posición inicial, el destino, el tamaño de cada celda y la orientación inicial.
3. Permitir movimientos en cuatro direcciones: arriba, abajo, izquierda y derecha.
4. Generar, en cada paso, el conjunto de movimientos posibles.
5. Descartar posiciones fuera de la grilla, obstáculos, zonas prohibidas y posiciones ya visitadas.
6. Calcular la distancia Manhattan desde cada candidato hasta el destino.
7. Seleccionar el candidato de menor distancia y aplicar un criterio fijo de desempate.
8. Registrar la ruta recorrida y la cantidad de movimientos.
9. Finalizar al alcanzar el destino, al quedar sin candidatos válidos o al superar el máximo de pasos.
10. Informar claramente el resultado y la causa de finalización.
11. Traducir la ruta a instrucciones de orientación y desplazamiento para el robot.
12. Probar el programa con los tres mapas provistos y analizar sus resultados.

3. Reglas del problema

- El valor 0 representa una celda libre.
- El valor 1 representa un obstáculo.
- El valor 2 representa una zona prohibida.
- El robot no puede visitar dos veces la misma celda durante una ejecución.
- La estrategia greedy no puede retroceder ni revisar decisiones anteriores.
- El programa debe establecer un límite máximo de pasos para garantizar su finalización.

4. Función de selección greedy

Para cada posición candidata (fila, columna) se utilizará la distancia Manhattan al destino:

```
distancia = abs(fila - filaDestino) + abs(columna - columnaDestino)
```

Se seleccionará el candidato con menor distancia. En caso de empate deberá utilizarse un orden fijo y documentado, por ejemplo: ARRIBA → DERECHA → ABAJO → IZQUIERDA.

5. Criterio de finalización

DESTINO_ALCANZADO	El robot llegó a la celda destino.
BLOQUEADO	No existen candidatos válidos para continuar.
LIMITE_DE_PASOS	Se alcanzó el máximo de pasos permitido.

6. Traducción de la ruta a instrucciones

La ruta de posiciones deberá transformarse en instrucciones físicas considerando la orientación actual del robot y el tamaño de cada celda.

Ejemplo:
Ruta: (0,0) → (0,1) → (1,1)
Orientación inicial: ESTE
Tamaño de celda: 0,50 m
Instrucciones: AVANZAR 0,50 m → GIRAR DERECHA 90° → AVANZAR 0,50 m

7. Casos de prueba

Mapa 1 - Solución directa: El algoritmo debe alcanzar el destino mediante elecciones que reducen continuamente la distancia Manhattan.

Mapa 2 - Solución no óptima: El algoritmo debe alcanzar el destino, pero utilizando una ruta más larga que otra ruta posible. Debe explicarse por qué la selección local no garantiza optimalidad global.

Mapa 3 - Fracaso de la estrategia: El algoritmo se aproxima al destino y queda bloqueado. Existe una ruta válida, pero requiere alejarse temporalmente del objetivo. El resultado esperado es informar el bloqueo, no modificar greedy para forzar la solución.

8. Entregables

- Archivo tp_greedy_apellido.py con la implementación completa.
- Archivos de log o capturas de las tres ejecuciones.
- Representación textual o gráfica de cada ruta.
- Secuencia de instrucciones generada para el robot.
- Informe técnico de 1 a 2 páginas.
- Demostración simulada y, cuando sea posible, demostración física supervisada.

El informe deberá explicar la función de selección, el criterio de factibilidad, el mecanismo de desempate, los resultados de cada mapa y las limitaciones observadas.

9. Criterios de evaluación

Criterio	Indicadores	Puntaje
Lectura del mapa	Carga y valida correctamente la grilla y los metadatos del JSON.	15
Algoritmo greedy	Genera candidatos válidos, calcula la distancia y aplica selección y desempate correctamente.	30
Finalización	Detecta destino, bloqueo y límite de pasos sin ciclos indefinidos.	15
Análisis de casos	Explica éxito, ruta no óptima y fracaso de la estrategia.	20
Integración con el robot	Genera instrucciones coherentes según orientación y tamaño de celda.	10
Código e informe	Código modular, nombres descriptivos y explicación técnica con palabras propias.	10

TOTAL: 100 puntos

10. Seguridad

La ruta debe probarse primero en el simulador. La demostración con robot real será opcional, a velocidad reducida y siempre supervisada por el docente. El área de trabajo deberá estar despejada y delimitada.